MORTIS

Anonymous Authors[1]

ABSTRACT

Existing agent serving systems exploit the predictability of agentic workloads to guide KV-cache scheduling but only at a binary prefix–suffix granularity. We observe that agent prompts have finer schedulable structure, a property we call *intra-prompt heterogeneity*: they can be decomposed into multiple heterogeneous components that differ in sharing pattern and lifetime. This property creates opportunities to enhance KV-cache management by exposing more prefix reuse and scheduling KV-cache more precisely. Meanwhile, frameworks that let LLM invocations be declared as language constructs emit prompts programmatically, making intra-prompt heterogeneity analyzable from program code. To this end, we present MORTIS, a compiler-serving co-design that recovers and exploits intra-prompt heterogeneity for efficient KV-cache management. Our evaluation shows that MORTIS reduces average JCT by 1.40× and average TTFT by 1.62× on geometric mean over vanilla SGLang, and by up to 1.90× and 4.49×. On a mixed workload, it raises throughput by 1.51× on geometric mean and by up to 1.86×.

1 INTRODUCTION

Agent applications accomplish tasks through a series of dependent LLM requests that often share long contexts. A recent characterization reports that 85–97% of prompt tokens are shared across requests (Srivatsa et al., 2025). Serving one LLM request involves a prefill phase that computes the key-value (KV) cache for the prompt, followed by autoregressive decoding. Prefill is particularly expensive for long prompts because attention cost grows quadratically with prompt length, but the extensive context sharing in agent applications creates substantial opportunities to avoid redundant prefill computation by reusing the KV cache. However, KV cache reuse cannot be taken for granted: limited GPU memory means that not every reusable KV cache can be retained. Agent serving systems must therefore manage the KV cache carefully, as losing reusable entries is costly: once the KV cache thrashes, per-request prefill time can grow by 6.8× and generation throughput can drop by up to 66% (Yuan et al., 2026).

Fortunately, agent workloads are far more predictable than generic chat traffic. Proir serving systems exploits such predicablity to schedule KV-cache. KVFlow (Pan et al., 2025) derives each agent's steps-to-execution from the workflow graph to guide eviction and prefetching; Helium (Wadlom et al., 2026) analyzes the workflow to order LLM calls and prewarm static prefixes; PBKV (Zheng et al., 2026) and CacheScout (Zhang et al., 2026) predict upcoming agent invocations with trained model decides which caches to retain. All of them inherits the binary view like prefix caching's in scheduling: a reusable prefix followed by an opaque suffix, and caching decisions are made at the granularity of that prefix. What lies inside the prompt and how differently they behaves are largely invisible to the scheduler.

We observe that agent prompts have finer schedulable structure than this binary view. A prompt is a composition of heterogeneous components whose **sharing pattern** and **lifetime** differ. We call this property **intra-prompt heterogeneity**. The fact-checking agent in Figure 1 exhibits it: **(i)** *sharing pattern*: *claim* is shared by all four calls, whereas *evidences* is shared only by calls C and D; **(ii)** *lifetime*: *claim* and *evidences* live for the whole session, with *evidences* growing every iteration, whereas *query* and *search_result* are replaced in each loop iteration.

This property opens two optimization opportunities (Figure 1). **First**, more prefix caching can be exposed by laying out components in prompts. Because prefix reuse stops at the first different token, placing stable, widely shared components at the front keeps a divergent token from cutting off the reusable prefix, such as placing shared *claim* at the beginning. Among the agents we evaluated (§5), 51–92% of the shared tokens cannot reuse the KV-cache because of such invalidation. **Second**, the KV-cache can be scheduled more precisely. Rather treating prompts as a monolithic unit, they can be scheduled at the granularity of components. For example, evicting KV-cache for *query* and *search_result* after their last use.

Exploiting these opportunities requires discovering intra-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

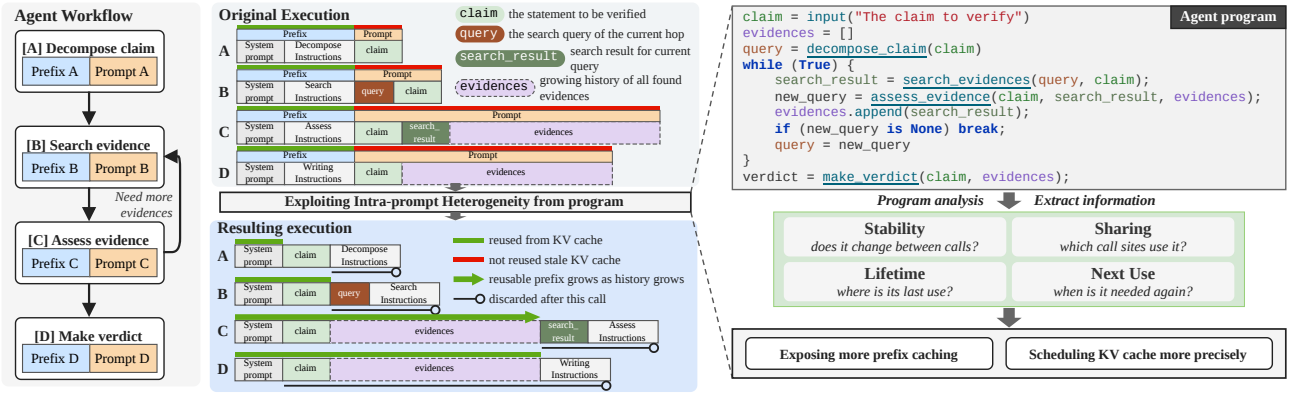


Figure 1. An example of a fact-checking agent. Prompts can be decomposed into heterogeneous components. This information can be derived from program itself and used to guide efficient KV-cache scheduling.

prompt heterogeneity. **We argue that intra-prompt heterogeneity information resides in agent program itself, which can be discovered with compiler techniques.** Agent frameworks like byLLM, NOOA, DSpy (Furgale et al., 2026; Khattab et al., 2024; Dantanarayana et al., 2025) expose LLM invocations through programming abstractions, letting developers declare LLM invocations as functions with typed parameters. Then the framework runtime generates the prompts from this declarations and the argument values. Consequently, components within prompts are mapped to each variable, making sharing patterns and lifetimes encoded in program itself. Intra-prompt heterogeneity is therefore analyzable. As Figure 1 shows, the program code itself reveals this heterogeneity and the derived information guides more efficient KV-cache scheduling.

To this end, we present MORTIS, a compiler-serving co-design that recovers and exploits intra-prompt heterogeneity. MORTIS uses *adaptive prompt re-layout* and *proactive KV-cache scheduling* to expose more prefix-cache reuse and to schedule scarce cache memory more efficiently. However, exploiting intra-prompt heterogeneity raises several challenges. **First, the comprehensive recovery needs both code information and actual runtime behaviors.** Static analysis can provide basic structure, but knowledge on behavior comes from runtime. MORTIS bridge the gap by combining information from both sides with its compiler-serving co-design. **Second, prompt layout is local to each call-site, but maximizing prefix reuse requires coordination across the whole agent.** MORTIS uses a five-level tagging system and analyze based on workflow to determine the final layout to maximize prompt reuse. **Third, KV-cache scheduling consumes the resourses it tries to save.** Retention and prefetching hold memory that other requests could use, yet evicting too eagerly forces recomputation. MORTIS protects the prefixes most likely to be reused within a fixed memory budget, and prefetches only into memory reclaimed from dead KV-cache.

In summary, we make the following contributions: **(i)** We identify *intra-prompt heterogeneity* in agent workloads and the further optimization opportunities it offers. **(ii)** We propose a methodology that uses compiler techniques to recover intra-prompt heterogeneity and guide efficient KV-cache management with that property. **(iii)** We design and implement MORTIS, a compiler-serving co-design that guides KV-cache scheduling with the recovered intra-prompt heterogeneity. Our evaluation shows that MORTIS reduces average JCT by 1.40× and average TTFT by 1.62× on geometric mean over vanilla SGLang, and by up to 1.90× and 4.49×. On a mixed workload, it raises throughput by 1.51× on geometric mean and by up to 1.86×.

2 BACKGROUND

2.1 Agent frameworks with LLM calls as programming abstractions

A class of agent frameworks exposes model-backed computation through typed programming interfaces. Examples include byLLM's meaning-typed programming (MTP) abstraction (Dantanarayana et al., 2025), DSPy's Signatures and Modules (Khattab et al., 2024), and NOOA's typed Python methods (Furgale et al., 2026). The developer declares a function or signature describing the desired behavior, inputs, and return type, and invokes it like an ordinary call within the program's control flow. At each invocation, the framework runtime lowers the declaration and its argument values into model-facing messages.

2.2 KV-cache scheduling in LLM agent serving

LLM agent executions create an enormous KV-cache footprint, which should be carefully managed to avoid repeated computation. PagedAttention manages these states in share-

able memory blocks (Kwon et al., 2023), while SGLang indexes cached token sequences in a radix tree (Zheng et al., 2024). An incoming request reuses the longest cached prefix of its prompt and computes KV for the remaining suffix.

Hierarchical caching systems such as CachedAttention (Gao et al., 2024), Mooncake (Qin et al., 2025), and SGLang HiCache (SGLang Contributors, 2026) extend the KV-cache beyond device memory with host memory and, optionally, disk storage. KV evicted from the device can remain in the host tier and be reloaded on reuse, which can be cheaper than recomputation. However, reloading is not free. Transfers add latency unless they are overlapped with other work. Moreover, the host tier itself is finite. Offloading only relocates cache-capacity pressure but does not eliminate it.

Under memory pressure, engines evict KV at the granularity of whole requests or cached prefixes. SGLang evicts unused cached prefixes by recency (Zheng et al., 2024); Continuum (Li et al., 2025) pins KV across tool calls for a predicted duration; KVFlow (Pan et al., 2025) evicts first the KV farthest from execution, measured as steps-to-next invocation on the agent step graph; CacheScout (Zhang et al., 2026) prefetches and protects agent prompt anchors using transitions learned online; PBKV (Zheng et al., 2026) manages eviction and protection with predicted reuse from workflow history; Pythia (Yu et al., 2026) reconstructs the agent workflow from historical execution traces and uses the predicted execution graph for speculative KV-cache staging and eviction, request scheduling, and autoscaling across serving instances.

3 MOTIVATION

Runtime-lowered prompts have similar structures Although these agent frameworks differ in surface syntax, they lower LLM invocations in similar ways. They declare LLM invocations as functions or function-like declarations, and the framework serializes that declaration together with the runtime arguments into a single prompt.

An example in Figure 2 demonstrates this. ByLLM, DSPy, and NOOA lowered prompts carry the same five kinds of components: framework scaffolding, task instruction, parameter descriptions, argument values, and output schema. The only difference is their relative position. Moreover, `argument values` are all emitted in declaration order.

The component order carries no intent The order in which these five components appear is decided solely by the framework runtime, without any program-specific knowledge. Figure 2 shows that the three representative frameworks arrange the same five components differently for the same invocation, which suggests that no particular order is required by the task. Even when argument values appear in declaration order, they do so simply because the runtime follows the parameter list, whose order was chosen by human developers.

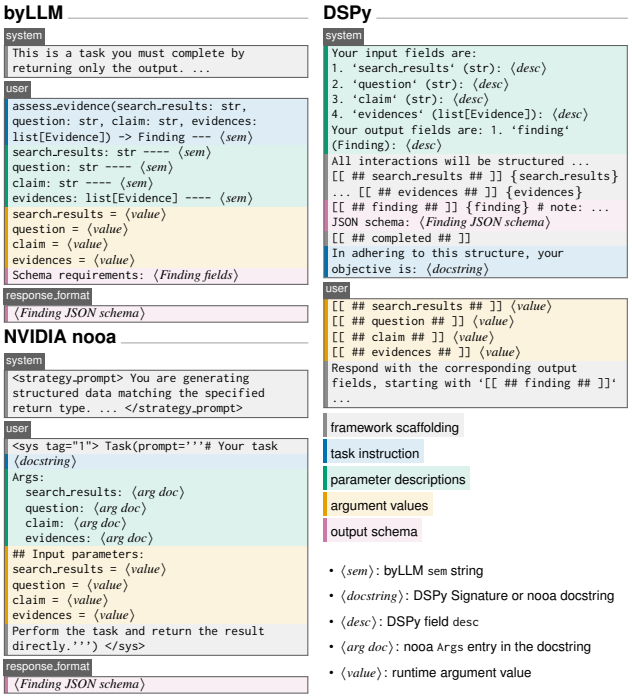


Figure 2. Prompts lowered by three frameworks for the same call share five kinds of components; only their order differs.

Prompt components are heterogeneous A prompt programmatically generated by an agent framework is not prose. Instead, it has inherent heterogeneity. The runtime constructs the prompt from different sources, such as program constants, runtime values, and external inputs. Therefore these components have different sharing scopes, lifetimes, and evolution patterns. As we discussed in §1 for Figure 1, this heterogeneity affects how the KV-cache can be managed effectively.

4 DESIGN

In this section, we present the two-tier compiler-serving co-design of MORTIS. As shown in Figure 3, *agent descriptor* is generated by the combination of analyzing agent program and observing runtime behaviors. Server side then exploits the derived descriptor to schedule the KV-cache efficiently, using *adaptive prompt re-layout* (§4.2) and *proactive KV-cache scheduling* (§4.3).

4.1 Agent Descriptor

Agent descritor contains the metadata for scheduling. Static analysis provides it a skeleton and the server refines the

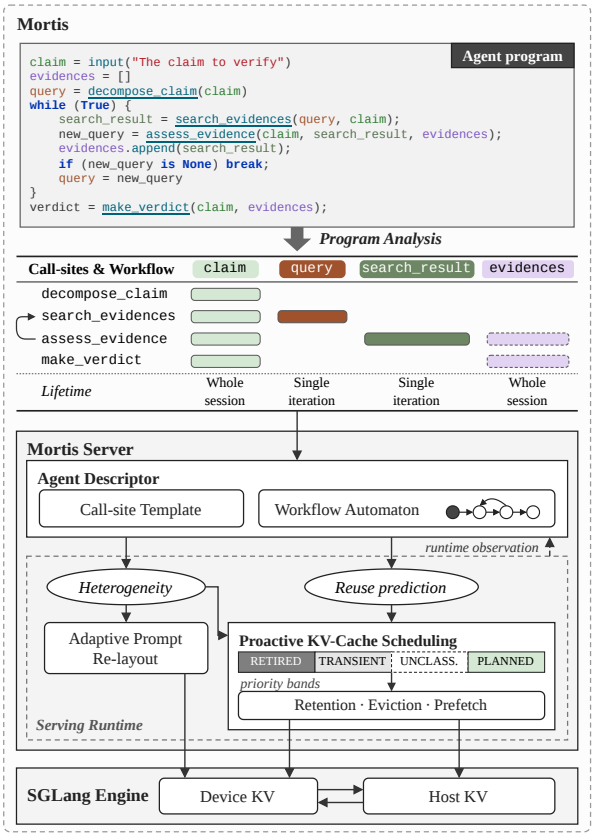


Figure 3. Architecture overview of MORTIS.

weights with runtime observations. An agent descriptor contains two parts: (i) **Call-site Template:** It provides a heuristic of the components' lifetime, sharing scope and evolution pattern. (ii) **Workflow Automaton:** It contains the program-level structure of an agent, including workflow dependencies, execution time estimation and transition probabilities.

MORTIS traverses the agents type-checked AST to identify LLM call sites and their relationships. It then classifies each parameter's provenance and lifetime to construct the skeleton of the agent descriptor.

4.1.1 *Call-site Template*

As discussed in §3, the components of a programmatically generated prompt can be classified into the following two categories: (i) **Invariant Specifications:** the components that are fixed once the program is compiled, such as the system prompt, response schema, and the tool declarations. They are rendered from the compiler's own intermediate representation and stay constant. (ii) **Dynamic Bindings:** because LLM calls are declared as ordinary functions, the arguments of a call site are supplied at runtime. These runtime values are the dynamic bindings.

We therefore define the call-site template as

$$\mathcal{T}_c = \langle \mathcal{I}_c,\ \mathcal{B}_c,\ \mathcal{H}_c \rangle,$$

where $\mathcal{I}_c$ is the invariant specifications, $\mathcal{B}_c$ is the dynamic bindings, and $\mathcal{H}_c$ is the heterogeneity annotation over those bindings. All three are recovered by static analysis.

Heterogeneity annotation. For each dynamic binding, $\mathcal{H}_c$ records both how its value is shared and how long that value stays valid:

$$\mathcal{H}_c :\ \mathcal{B}_c \longrightarrow \mathcal{E} \times \mathcal{S}.$$

The sharing pattern component $\mathcal{E}$ classifies a binding by its value provenance. We distinguish $\mathcal{E}$ into five classes, listed from the most to the least reusable: **(i) CONST:** the value is fixed by the program text and is identical at every call. **(ii) COPY:** the value is the same as the value another call site carries, so the bytes are already resident once that call site has run. **(iii) RESP:** the value comes from the reply produced by another call site. **(iv) EXTEND:** the binding is an append-only context, and the previous value remains a prefix of the new one. **(v) VOLATILE:** the value is freshly produced at each call, or its origin lies outside what the analysis can see. We write ρ for the rank of a class in the order above, which expresses how much of a binding's bytes an earlier call can already supply.

The lifetime component $\mathcal{S}$ names the program scope whose completion invalidates a binding's value ($\mathcal{S} = \emptyset$ meaning that the value survives for the whole session once produced).

The evolution component $\mathcal{E}$ governs which bytes a later prompt can reuse, while the lifetime component $\mathcal{S}$ governs when cached bytes can be given up.

4.1.2 *Workflow Automaton*

An agent typically consists of a sequence of dependent LLM calls whose workflow structure is encoded in the program. Concretely, we abstract an agent as a time-annotated probabilistic finite-state machine (TPFM). We formulate the TPFM as:

$$\mathcal{A} = (Q, P, q_0, F, \mathcal{T}),$$

where: (i) Q is the set of states. Each state $q \in Q$ is labeled by an LLM callsite; (ii) P denotes the transition probabilities between states. (iii) $q_0 \in Q$ is the entrance callsite. (iv) F specifies the termination probability of the agent at each state. (v) $\mathcal{T} = \left\{ D_q^{\text{exec}}, D_{q,q'}^{\text{gap}} \right\}$ denotes the timing annotations associated with states and transitions, where D_q^{exec} is the execution-time distribution of the LLM callsite represented by state q, and $D_{q,q'}^{\text{gap}}$ is the inter-call gap distribution between the completion of callsite q and the invocation of its successor q'.

Static analysis fixes the shape of this automaton, while its weights come from runtime logs. The states, the entry, the transitions among them, and final states are recovered from code. $\mathcal{T} = \left\{D_q^{\text{exec}}, D_{q,q'}^{\text{gap}}\right\}$ and P are often unknown at compile time and model-dependent. The server observes these online and accumulates them across sessions, updating the statistical model of the agent consistently. Together the two halves give the server a forward-looking view of a session.

4.2 Adaptive Prompt Re-layout

Prefix reuse stops at the first byte that differs from a cached prompt. The message a framework emits for a call site follows the declaration order of the function, which may place a changing value ahead of values that later calls could reuse. MORTIS's adaptive re-layout permutes the components of each call message so that the bytes most likely to be resident already come first. The descriptor $\mathcal{T}_c$ (§4.1.1) and $\mathcal{A}$ (§4.1.2) determine a partial order over the components, and the server completes it into a layout with the token lengths it observes at runtime.

4.2.1 Movable Components

First, the dynamic bindings $\mathcal{B}_c$ can be permuted freely. The compiler lowers each argument into a self-contained component labeled by its parameter name, and the framework places these components in declaration order only because that is the order of the argument list; re-layout may therefore choose any order of the bindings.

Second, part of the invariant specification $\mathcal{I}_c$ is movable as well. $\mathcal{I}_c$ is treated as one block abstraction in §4.1.1, but it has internal structure. It consists of the system prompt, the response schema, the tool schema, and the instruction specification (the task description and parameter descriptions generated by the compiler). Among them, the system prompt stays universally constant, and the two schemas are consumed by the model separately. The instruction specification is movable.

MORTIS therefore operates on the order of bindings and the placement of instruction specification. §4.2.2 describes how MORTIS chooses a layout within this space.

4.2.2 Layout Decision

Each call site sorts the bindings it carries by a program-wide key. Generally, *shared* bindings, which are carried by multiple call sites, are placed at the front, followed by the bindings carried by a single call site. Algorithm 1 summarizes the procedure.

Sorting private bindings. The bindings carried by a single call site are sorted in ascending lexicographic order of

$$K(b) = \big(\textit{scoped},\ \rho,\ \textit{depth},\ -\textit{length}\big),$$

whose components are defined as follows: **(i)** *scoped*: the lifetime of b; 0 if it is session-wide ($\mathcal{S} = \emptyset$) and 1 otherwise, so bindings that survive scope boundaries precede those reset by them. **(ii)** ρ: the stability rank defined in §4.1.1, from 1 for CONST to 5 for VOLATILE, so the more reusable class leads. **(iii)** *depth*: The number of transitions to the first call site that carries it from entrance, so values that enter a session earlier lead. **(iv)** *length*: the token length of the value of b, observed by the server; negated so longer bindings precede.

Sorting shared bindings. The order of shared bindings is derived from program-level structures. For every transition from q to q', Static Analysis identifies how each shared binding of the two call sites evolves. We categorize these patterns into three groups, ordered as follows: (**i**) Both call sites carry it and its value does not change between the two calls. (**ii**) Both call sites carry it and the program only appends to its value. (**iii**) Program rewrites it between the two calls, or only one of the two call sites carries it.

MORTIS collects these requirements over all transitions and orders the shared bindings to satisfy them; when several bindings can go next, the one with the smallest (***-length***, ***depth***) is placed first. Bindings with conflicting requirements are clustered and sorted by the same key.

Placing the specification. The *instruction specification* is lowered from code by the compiler and is unique to each call site. Emitting it first would prevent prefix sharing among different call sites. After sorting, MORTIS moves the specification to the end when the leading bindings are shared, so that sibling call sites can share larger common prefix. A switch keeps the specification in front when moving it is suspected of changing the model's behavior.

4.3 Proactive KV-Cache Scheduling

While adaptive prompt re-layout improves prefix reuse only by reshaping the request, MORTIS's KV-cache scheduling proactively arranges the placement of KV-cache entries. Using the agent descriptor, the server decides for every served KV-cache sequence what to keep, what to give up, and what to prefetch.

4.3.1 Reusable Head and Volatile Tail

Since re-layout already places the stable and shared bindings at the front, a served prompt naturally divides into two segments: a reusable head and a volatile tail.

The reusable head consists of two parts: a *static prefix* and

Algorithm 1 Layout decision algorithm

Require: Descriptor $(\{\mathcal{T}_c\}_{c\in C}, \mathcal{A})$, lengths ℓ, switch *pin*
Ensure: Layouts $\{L_c\}_{c\in C}$

$B_{\text{sh}} \leftarrow \{b : |\{c \in C : b \in \mathcal{B}_c\}| > 1\}$; $R \leftarrow \emptyset$
for all $e = (q, q') \in \textsc{Edges}(\mathcal{A})$ **do**
  $B_e \leftarrow B_{\text{sh}} \cap (\mathcal{B}_q \cup \mathcal{B}_{q'})$
  $t_e(b) \leftarrow 1$ if *unchanged*, 2 if *append-only*, else 3
  $R \leftarrow R \cup \{(b, b') \in B_e^2 : t_e(b) < t_e(b')\}$
$J(b) \leftarrow (-\ell(b), \mathit{depth}(b))$; $J_{\min}(S) \leftarrow \min_{b\in S} J(b)$
$D \leftarrow \textsc{ClusterConflicts}(B_{\text{sh}}, R)$
$\langle S_1, \ldots, S_k\rangle \leftarrow \textsc{TopoSort}(D, J_{\min})$ ▷ Min-key first
$\pi_{\text{sh}} \leftarrow \textsc{Sort}(S_1, J) \| \cdots \| \textsc{Sort}(S_k, J)$
for all $c \in C$ **do**
  $\pi_c \leftarrow \langle b \in \pi_{\text{sh}} : b \in \mathcal{B}_c\rangle \| \textsc{Sort}(\mathcal{B}_c \setminus B_{\text{sh}}, K)$
  $s_c \leftarrow \textsc{InstructionSpec}(\mathcal{I}_c)$
  $L_c \leftarrow \langle s_c\rangle \| \pi_c$
  if $\neg$*pin* $\wedge$ $(\mathcal{B}_c \cap B_{\text{sh}} \neq \emptyset)$ **then** $L_c \leftarrow \pi_c \| \langle s_c\rangle$
return $\{L_c\}_{c\in C}$

a *session-shared extension*: (i) **Static Prefix** is what the call site emits without runtime inputs, and its value outlives a single session. It includes the invariant specifications $\mathcal{I}_c$ together with the CONST bindings that re-layout has placed right behind $\mathcal{I}_c$. (ii) **Session-shared Extension** is the bindings behind the static prefix that a later call of the same session will emit again identically. MORTIS determine the reusable head by analyzing $\mathcal{H}_c$, $\mathcal{A}$ and the decided prompt layout jointly.

4.3.2 Next-Call Prediction

Our prediction focuses on the following three aspects: (i) The probability of the arrival. (ii) When the call will arrive. (iii) Which prefix of its prompt can be speculatively constructed.

Arrival probability. We estimate the probability of the next call by a loop-tagged table. MORTIS adopts Bourdoncle-style (Bourdoncle, 1993) weak topological ordering to discover all loops within the agent based on the emitted workflow graph. Each call-site q can be located by a chain of loops enclosing it; for each call-site's invocation, the server marks it with a tuple $\zeta = \{i_1, i_2, \ldots, i_n\}$, where i_k is the current iteration index of the k-th enclosing loop, outermost first. We define the table as:

$$\mathcal{P}(q' \mid q, \zeta) \qquad \mathcal{F}(q, \zeta)$$

Where $\mathcal{P}$ stands for the probability that a call-site q in context ζ is followed by a call-site q'; $\mathcal{F}$ stands for the probability that the session ends at call-site q under context ζ. With an empty ζ (no loop context), $\mathcal{P}$ and $\mathcal{F}$ coincide with P and F.

The table is a frequency count that the server updates across sessions. Let q and ζ denote the call-site and loop context of the preceding call in the same session. When a call at q' arrives, the server increments the count of outcome q' in row (q, ζ); when the session closes at q, it increment termination count in also in row (q, ζ); Each observation is also counted in the row of every suffix of ζ, down to the empty tuple, so that for the same q the table holds one row per level of context. After a call at q lands, we compute the probability of each outcome o by Witten–Bell interpolation from the empty context ζ_0 to the full context $\zeta_m = \zeta$, where o ranges over $\text{succ}(q)$, the static successors of q plus a virtual outcome end for session termination:

$$\hat{P}_k(o) = \lambda_k \frac{c_k(o)}{n_k} + (1 - \lambda_k)\, \hat{P}_{k-1}(o), \qquad k = 0, \ldots, m,$$

$$\hat{P}_{-1}(o) = \frac{1}{|\text{succ}(q)|}, \qquad \lambda_k = \begin{cases} \dfrac{n_k}{n_k + d_k} & n_k > 0, \\ 0 & n_k = 0, \end{cases}$$

where ζ_k is the suffix of ζ that keeps only the innermost k indices, and the three counts are read from the row (q, ζ_k) of the table: $c_k(o)$ is how many times outcome o followed a call at q in context ζ_k, $n_k = \sum_o c_k(o)$ is the number of observations in that row, and d_k is the number of distinct outcomes with $c_k(o) > 0$.

Then,

$$\mathcal{P}(q' \mid q, \zeta) = \hat{P}_m(q') \qquad \mathcal{F}(q, \zeta) = \hat{P}_m(\text{end})$$

A level with no observation is skipped. The weight λ_k grows with the support of a level, so a well-observed iteration dominates the prediction while a sparse one defers to the pooled behavior of q.

Arrival time. The time of arrival is conservatively approximated based on recorded statistics. The workflow automaton records $D^{\text{gap}}_{q,q'}$ per transition: each time a call at q' arrives, the server appends the interval between the reply of the preceding call at q and this arrival. The interval is the agent's own time spent in its code between two calls.

We estimate the gap before the next call as:

$$\hat{g} = t_{50} - \tfrac{1}{2}(t_{90} - t_{50})$$

where t_{50} and t_{90} are the 50th and 90th percentiles of the gap samples on the transition (q, q'); the predicted call is then expected $\hat{g}$ after the reply of the current call. The second term ensures that a transition with a more dispersed gap distribution receives a tighter estimate.

Prefix reconstruction. As indicated in §4.3.1, the prompt of the known successor q' is reconstructible, or at least partially reconstructible. MORTIS builds its prompt in the

layout $L_{q'}$, using the values that the session already carries. The construction processes bindings in the order of $L_{q'}$ while consulting their evolution class, deducing values accordingly. The construction stops at the first binding with no known origin. Then the matched prefix is sent to the kv-planner for speculative preparation.

4.3.3 Lifecycle Bands

As we discussed in §3, different components have different lifetimes, sharing scopes, and evolution patterns. Therefore, relying on recency-based LRU would lead to suboptimal performance. MORTIS adopts a banded priority and steers KV-cache eviction with it.

Bands. We categorize KV-cache entries into four bands, from first evicted to last: (1) **RETIRED:** the volatile tails of prompts belonging to completed sessions. (2) **TRANSIENT:** the volatile tail of a served prompt of a live session. The session that produced them will not present them again. (3) **UNCLASSIFIED:** entries that analysis cannot categorize, such as the prompts of a program that was never registered. They keep the engine's default treatment. (4) **PLANNED:** prefixes that a predicted call will reuse. Inside this band entries are ordered by the expected value of keeping them resident, defined next.

Eviction empties the lowest band first and falls back to recency only within a band, so KV-cache that will likely be reused leaves memory later. On the host tier it keeps LRU eviction except that retired segments go first, so a head the device gave up under pressure remains reloadable while it may still be needed. Additionally, KV-cache entries marked as *TRANSIENT* are not admitted into the host tier, as they wouldn't be reused.

Band transitions. Bands are assigned at three moments of a session's lifecycle. (1) **When a served prompt enters the cache**, MORTIS splits it at its reusable head (§4.3.1). The volatile tail and the reply generated after it become *TRANSIENT*; the head stays *UNCLASSIFIED* until the next prediction refresh finds a successor call that starts with it, which promotes it to *PLANNED*. (2) **When a later call arrives**, the transition it takes may rewrite bindings that earlier heads carried. Each affected head is cut back to the prefix that survives the rewrite, and the bytes past it are demoted to *TRANSIENT*. (3) **When the session ends**, the volatile tails of every prompt it served drop to *RETIRED*. Their heads stay *UNCLASSIFIED* and age out by recency, since other sessions of the same program may still share them.

These bands are mirrored directly into the radix tree's eviction order. The band is the primary key of the eviction heap and recency breaks ties.

4.3.4 Retention and Prefetch

Based on the prediction, the kv-planner of MORTIS conducts protection. Its duty is to ensure that the required prefix resides on the device, or at least on the host. Each prediction is summarized into a tuple of reconstructed prefix, arrival probability and estimated deadline:

$$\textit{Prediction} = (\pi_{q'},\ \mathcal{P}(q' \mid q, \zeta),\ t_{\text{reply}} + \hat{g})$$

where $\pi_{q'}$ is the prefix reconstructed for q' and t_{reply} is the time the current call replied. Predictions are refreshed at every call, and are dropped when the predicted calls arrive or expire. Protection acts on this tuple in two ways: *Retention* keeps $\pi_{q'}$ from leaving the device, and *prefetch* restores the part of $\pi_{q'}$ that has already been demoted to the host.

Retention. The planner ranks the predicted reusable KV-cache entries by the value of keeping $\pi_{q'}$ resident at every refresh:

$$\mathcal{D}(q') = \frac{\mathcal{P}(q' \mid q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}}{\max(t_{\text{reply}} + \hat{g} - t_{\text{now}},\ \tau_{\min})}$$

where $|\pi_{q'}|_{\text{cached}}$ is the number of tokens of $\pi_{q'}$ cached (on either host or device) and $\tau_{\min}$ is a floor that keeps a call due now from dominating. MORTIS's kv-planner sorts all predictions by $\mathcal{D}$ in descending order, and admits them into *PLANNED* until the band covers a fixed fraction of the device tier. Within the band, their $\mathcal{D}$ add up on every radix cache node, so that a shared head outranks a private one.

Prefetch. When part of $\pi_{q'}$ has been demoted to the host, the planner prefetches it back to the device ahead of the estimated arrival time. A prefetch release deadline is defined as:

$$t_{\text{release}} = t_{\text{reply}} + \hat{g} - \frac{|\pi_{q'}|_{\text{host}}}{B_{\text{h2d}}} - \frac{|\pi_{q'}|_{\text{uncached}}}{B_{\text{prefill}}}$$

where B_{h2d} and B_{prefill} are the measured host-to-device copy rate and prefill rate. Among released predictions, the earliest deadline goes first, with ties broken by expected saving:

$$\textit{Saving} = \mathcal{P}(q' \mid q, \zeta) \cdot |\pi_{q'}|_{\text{cached}}$$

A prefetch is released when the engine is idle or its deadline t_{release} approaches. It takes only free device memory, after reclaiming *RETIRED* and *TRANSIENT* entries. When no space is free, prefetching pauses but keeps touching its entries on the host so they do not age out by recency.

5 EVALUATION

In this section, we evaluate MORTIS on multiple representative agent workloads, and ablate each component.

5.1 Experimental Setup

5.1.1 *Testbed and Model*

We conduct experiments on one NVIDIA RTX 3090 GPU using Qwen3-14B-AWQ. We allocate 5.2 GB of GPU memory to the device KV-cache pool and 10 GB of CPU memory to the host KV-cache pool.

5.1.2 *Baselines*

We compare MORTIS against the following serving systems, together with the ablated variants of MORTIS. We also run each established serving system with MORTIS's adaptive prompt re-layout enabled. All baselines are reimplemented on MORTIS's SGLang fork with their original scheduling algorithms: (i) **Vanilla SGLang** (Zheng et al., 2024) uses SGLang's HiCache with LRU eviction. (ii) **CacheScout** (Zhang et al., 2026) identifies agents from their request prefixes and learns inter-agent execution transitions online. It uses the learned transition patterns to prioritize cached KV entries and prefetch the prefixes of predicted successor agents. (iii) **Continuum** (Li et al., 2025) retains each agent program's KV-cache for an adaptively selected time-to-live (TTL). The TTL is derived from observed inter-invocation delays and expected cache reuse, allowing the system to preserve reusable context while the agent executes tools. (iv) **KVFlow** (Pan et al., 2025) uses an Agent Step Graph to build a workflow-aware KV-cache management framework for agentic workloads. (v) **Static-Adapter** mirrors the adapter layer of DSPy (DSPy Contributors, 2026), which applies a fixed, program-agnostic layout to every call, hoisting the invariant components to the front. (vi) **MORTIS-KV-only** enables proactive KV-cache scheduling but disables adaptive prompt re-layout. (vii) **MORTIS-Relayout-only** enables adaptive prompt re-layout but replaces proactive KV-cache scheduling with LRU eviction.

5.1.3 *Workloads*

We evaluate MORTIS on four representative agent workloads, written in Jac byLLM (Dantanarayana et al., 2025): a HoVer-style (Jiang et al., 2020) fact-checking agent, a HumanEval (Chen et al., 2021) coding agent, a BFCL (Patil et al., 2025) tool-use agent, and a FinanceBench (Islam et al., 2023) financial-analysis agent. We further evaluate a mixed workload comprising all four agent types to assess MORTIS's end-to-end performance under concurrent, heterogeneous demand. The mixed workload assigns same lane count for each agent type;

5.2 End-to-End Performance of MORTIS

We test MORTIS under different numbers of concurrent workflows and on a mixed workload. Across these workloads, we compare average job completion time (JCT) and average time-to-first-token (TTFT), and throughput (prompt tokens served per minute) for the mixed workload.

Table 1. Performance on the mixed workload with 4 lanes for each agent.

System	Throughput (tokens(k)/min)	TTFT (s)	Hit rate (%) Device	Host	Total
Original prompt order					
Vanilla SGLang	90.8	41.6	6.2	0.9	7.1
Static-Adapter	103.6	33.7	4.1	4.2	8.3
CacheScout	92.3	41.7	6.4	0.2	6.6
Continuum	90.6	42.1	4.4	1.0	5.4
KVFlow	92.6	41.2	7.5	0.6	8.1
MORTIS-KV-only	105.7	35.3	7.6	6.9	14.5
With adaptive prompt re-layout					
MORTIS-Relayout-only	108.8	35.6	5.0	7.5	12.6
CacheScout	113.2	34.4	9.4	5.3	14.7
Continuum	110.8	34.8	4.8	7.1	11.9
KVFlow	116.0	34.1	10.8	10.1	20.9
MORTIS w/o prefetch	129.3	29.7	9.3	20.9	30.2
MORTIS	**132.3**	**29.0**	9.1	22.6	**31.6**

Performance on mixed workload We evaluate MORTIS under varying pressure on the mixed workload, measuring metrics over each system's steady-state window. Keeping all four agents at the same lane count, we vary the number of lanes each agent holds. As shown in Figure 4, MORTIS raises throughput up to 1.86× that of Vanilla SGLang with 2 lanes per agent and reduces average TTFT by up to 1.97×. Under low pressure adaptive prompt re-layout alone obtains nearly all of the gain, since device memory is still large enough to retain the reusable KV-cache. As pressure grows, the cache becomes contested and the gain of re-layout alone shrinks, while proactive KV-cache scheduling retains a clear advantage by protecting the reusable heads that LRU would evict. Under the heaviest pressure the two configurations converge toward the baseline, as the engine is completely saturated and there is no space to operate. Across the six lane counts, the geometric-mean throughput gain is 1.51×.

Table 1 reports the mixed workload at 4 lanes per agent. MORTIS raises throughput to 1.46× that of Vanilla SGLang and reduces average TTFT by 1.43×. Continuum and CacheScout perform close to Vanilla SGLang on all three metrics, and Static-Adapter stays within 1.14× of it. The gain of MORTIS comes from a higher cache hit rate in both device and host memory.

Per-workload sweeps Across the four workloads, MORTIS reduces average JCT by 1.06–1.90× and average TTFT by 1.14–4.49× compared with Vanilla SGLang (Figure 5), and a reduction of 1.40× in JCT and 1.62× in TTFT on

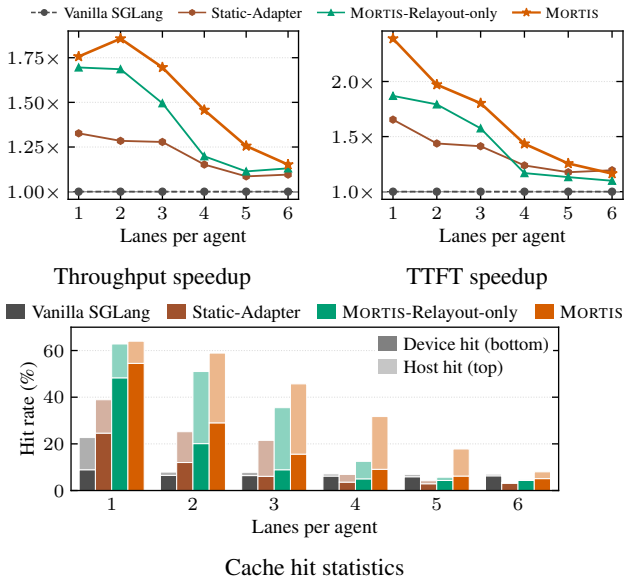


Figure 4. Throughput and average TTFT speedup over Vanilla SGLang, and cache hit status under varying load on the mixed workload.

Table 2. Task accuracy (%) of systems serving the original and the re-laid prompts, and p-values of a paired permutation test.

Workload	Original	Re-laid	Δ (pp)	p
Fact check	20.0	20.4	+0.4	0.95
Coding	92.1	91.8	−0.2	0.66
BFCL	7.2	6.4	−0.8	1.00
Finance	34.6	35.4	+0.8	0.82

geometric-mean. When concurrency is low, adaptive prompt re-layout accounts for nearly all of the gain. As concurrency grows, the gain of re-layout alone shrinks and proactive KV-cache scheduling carries it: it evicts volatile tails first and protects the reusable heads that predicted calls will need. On fact check, scheduling contributes from the lowest concurrency. On fact check and BFCL, and on finance at low concurrency, the TTFT reduction exceeds the JCT reduction, because a reused prefix skips its prefill, whereas JCT also includes decoding and tool execution, which the KV-cache does not shorten.

Except for a 1.09–1.22× improvement on BFCL at $c = 4$, Continuum, CacheScout and KVFlow perform close to Vanilla SGLang on both metrics. They exploit predictability between invocations but treat each prompt as monolithic units. In the original prompt order, volatile bindings often precede reusable ones, making prefixes hold little that later calls can reuse. MORTIS-KV-only, which schedules on the original prompt order, shows the same limit.

Task Quality We compare task accuracy between systems that serve the original prompts and systems that serve the re-laid prompts, using a paired permutation test over inputs (Table 2). Re-layout changes accuracy by at most 0.8 percentage points on any workload, with no consistent direction and no significant difference ($p \geq 0.66$). The performance gains of MORTIS therefore do not come at the cost of task quality.

5.3 Cache Efficiency of MORTIS

Figure 6 shows the cache hit rate of prompt tokens at the same concurrency levels as Figure 5. On coding and BFCL, adaptive prompt re-layout alone brings the hit rate at low concurrency to that of MORTIS: it places the stable and shared bindings at the front, where they form a reusable head that later calls hit. As concurrency grows, the hit rate of MORTIS-Relayout-only drops quickly, because LRU eviction does not distinguish reusable heads from volatile tails. With proactive KV-cache scheduling, MORTIS keeps a higher hit rate than MORTIS-Relayout-only as concurrency grows, and on coding its hit rate stays nearly flat. On fact check, MORTIS's hit rate exceeds that of MORTIS-Relayout-only from the lowest concurrency. Continuum and CacheScout stay close to Vanilla SGLang at every concurrency. MORTIS-KV-only stays well below MORTIS, since under the original prompt order the reusable head it can protect is short. The mixed workload shows the same pattern (Table 1, Figure 4).

5.4 Ablation Studies

We ablate MORTIS on the mixed workload by enabling its components one at a time (Table 1). MORTIS w/o prefetch keeps adaptive prompt re-layout and proactive KV-cache scheduling but disables prefetch. Figures 5 and 6 show MORTIS-KV-only and MORTIS-Relayout-only across concurrency levels.

Contribution of Each Mechanism Adaptive prompt re-layout alone raises throughput to 1.20× that of Vanilla SGLang and reduces average TTFT by 1.17×. Static-Adapter raises throughput by 1.14× and reduces average TTFT by 1.23× on the mixed workload. On the per-workload sweeps, however, it matches MORTIS-Relayout-only on finance, Vanilla SGLang on fact check, and degrades on BFCL (Figure 6 and Figure 5), because the effect of program-agnostic prompt layout hoisting depends on the program itself.

Adding proactive KV-cache scheduling brings throughput to 1.42× and the TTFT reduction to 1.40×, and lifts the hit rate from 12.6% to 30.2%. Prefetch adds little on this workload (1.46×, 1.43×), since under this load the device pool is rarely free enough for a prefetch to take place.

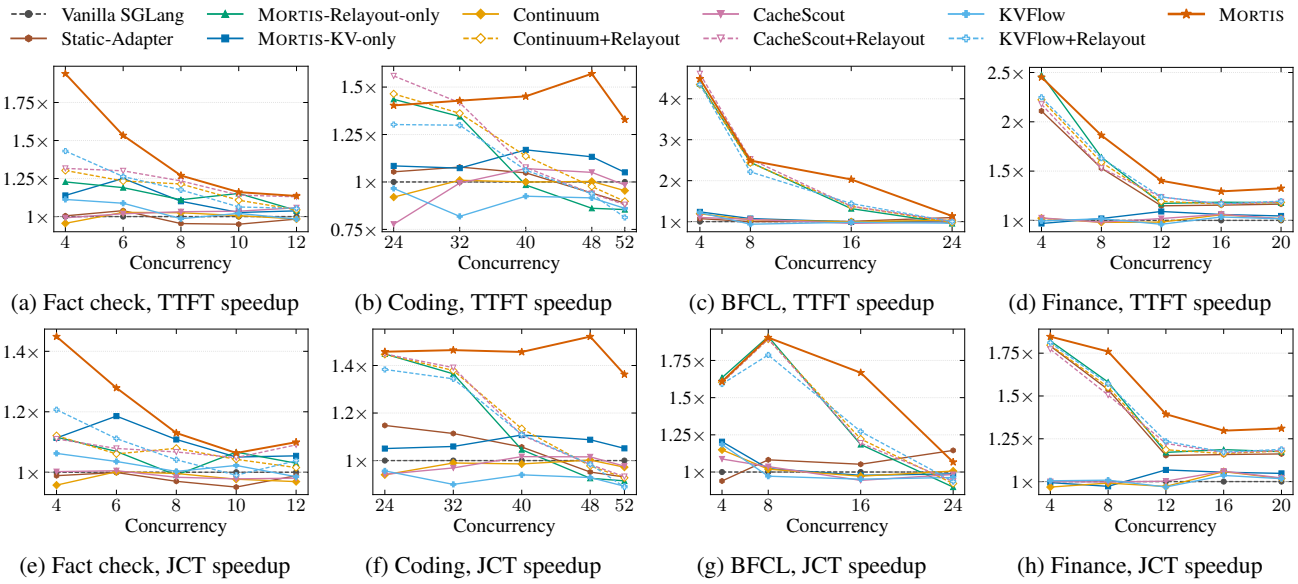


Figure 5. Average TTFT speedup (top) and average JCT speedup (bottom) over Vanilla SGLang under varying numbers of concurrent workflows.

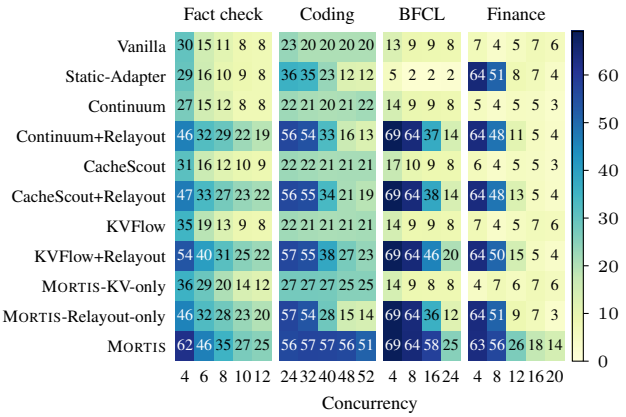


Figure 6. Cache hit rate (%) of prompt tokens under varying numbers of concurrent workflows.

Complementarity of Re-layout and Scheduling Adaptive prompt re-layout and proactive KV-cache scheduling depend on each other. Applied to the original prompt order, proactive KV-cache scheduling raises throughput by only 1.16× (MORTIS-KV-only); applied after re-layout, it raises throughput by 1.22× relative to MORTIS-Relayout-only. The combined 1.46× gain is larger than the sum of the two individual gains. Re-layout places the stable and shared bindings at the front, so a served prompt divides into a reusable head and a volatile tail, and scheduling gets opportunities to protect. Moreover, KV-cache scheduling and adaptive prompt re-layout are complementary. Applied on the re-laid prompts, CacheScout, Continuum and KVFlow all improve on MORTIS-Relayout-only (Table 1), and on the per-workload sweeps the gap widens with concurrency (Figure 5).

6 RELATED WORK

Frameworks for serving LLM agents. KVFlow (Pan et al., 2025) schedules KV by distance to the next invocation on an application-supplied agent step graph. Continuum (Li et al., 2025), CacheScout (Zhang et al., 2026), and PBKV (Zheng et al., 2026) learn retention signals from execution history, and Pythia (Yu et al., 2026) reconstructs the workflow from traces to drive cache staging, scheduling, and autoscaling. All of them manage whole requests or agent steps. MORTIS instead recovers the structure encoded in the program, and manages the KV-cache at intra-prompt granularity.

Prompt layout and reordering. Existing systems treat prompt layout as a fixed convention or delegate it to a model. SGLang's frontend reorders prompt statements to lengthen the shared prefix, but relies on GPT-4 to choose the order, which its authors note is not yet reliable (Zheng et al., 2024). DSPy's adapters emit components in one fixed order across all programs (DSPy Contributors, 2026), and NOOA argues for a three-region layout that places static context first (Furgale et al., 2026). These orders are chosen by convention or by an unsound rewrite. MORTIS derives the layout by systematically analyzing the program itself.

KV reuse beyond exact prefixes. Prompt Cache fixes module positions in a schema (Gim et al., 2024), and CacheBlend selectively recomputes tokens when concatenating cached chunks (Yao et al., 2025). Both relax positional

exactness to gain reuse, at the cost of soundness or extra recomputation.

7 Conclusion

The intra-prompt heterogeneity of programmatically generated prompts creates opportunities for optimization. MORTIS discovers these opportunities by static analysis assisted by runtime observations, and steers KV-cache management with this information through adaptive prompt re-layout and proactive KV-cache scheduling. Our evaluation demonstrates that MORTIS reduces average JCT by 1.40× and average TTFT by 1.62× on geometric mean over vanilla SGLang, and by up to 1.90× and 4.49×. On a mixed workload, it raises throughput by 1.51× on geometric mean and by up to 1.86×.

References

Bourdoncle, F. Efficient chaotic iteration strategies with widenings. In *Formal Methods in Programming and Their Applications*, volume 735 of *Lecture Notes in Computer Science*, pp. 128–141. Springer, 1993. doi: 10.1007/BFb0039704.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Dantanarayana, J. L., Kang, Y., Sivasothynathan, K., Clarke, C., Li, B., Kashmira, S., Flautner, K., Tang, L., and Mars, J. Mtp: A meaning-typed language abstraction for ai-integrated programming. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):1176–1204, 2025. doi: 10.1145/3763092.

DSPy Contributors. ChatAdapter: DSPy api reference. Online documentation, 2026. URL https://dspy.ai/api/adapters/ChatAdapter/. Accessed September 14, 2026.

Furgale, P., Klingler, S., Nolan, J., Staats, M., Di Lorenzo, G., Martinez Abad, E., Schüller, C., Dinu, R., Devoto, A., Berard, P., Kaplun, G., Sarafian, E., Roveri, R., Derczynski, L., and Silveira Cabral, R. NVIDIA-labs OO Agents: Native Python Object-Oriented Agents, 2026. URL https://arxiv.org/abs/2607.20709.

Gao, B., He, Z., Sharma, P., Kang, Q., Jevdjic, D., Deng, J., Yang, X., Yu, Z., and Zuo, P. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pp. 111–126, Santa Clara, CA, 2024. USENIX Association. URL https://www.usenix.org/conference/atc24/presentation/gao-bin-cost.

Gim, I., Chen, G., seob Lee, S., Sarda, N., Khandelwal, A., and Zhong, L. Prompt Cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, pp. 325–338, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.

Islam, P., Kannappan, A., Kiela, D., Qian, R., Scherrer, N., and Vidgen, B. Financebench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.

Jiang, Y., Bordia, S., Zhong, Z., Dognin, C., Singh, M., and Bansal, M. HoVer: A dataset for many-hop fact extraction and claim verification. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3441–3460, 2020.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *International Conference on Learning Representations*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. doi: 10.1145/3600006.3613165. URL https://arxiv.org/abs/2309.06180.

Li, H., He, R., Mang, Q., Zhang, Q., Mao, H., Chen, X., Zhou, H., Zhang, H., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live, 2025.

Pan, Z., Patel, A., Hu, Z., Shen, Y., Guan, Y., Li, W.-L., Qin, L., Wang, Y., and Ding, Y. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent

workflows, 2025. URL https://arxiv.org/abs/2507.07400.

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 2025.

Qin, R., Li, Z., He, W., Cui, J., Ren, F., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pp. 155–170, Santa Clara, CA, 2025. USENIX Association. URL https://www.usenix.org/conference/fast25/presentation/qin.

SGLang Contributors. HiCache system design and optimization. SGLang documentation, 2026. URL https://docs.sglang.io/docs/advanced_features/hicache_design. Accessed September 22, 2026.

Srivatsa, V., He, Z., Abhyankar, R., Li, D., and Zhang, Y. Preble: Efficient distributed prompt scheduling for LLM serving. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=meKEKDhdnx.

Wadlom, N., Shen, J., and Lu, Y. Efficient LLM serving for agentic workflows: A data systems perspective, 2026. URL https://arxiv.org/abs/2603.16104.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, New York, NY, USA, 2025. Association for Computing Machinery. doi: 10.1145/3689031.3696098. URL https://www.microsoft.com/en-us/research/uploads/prod/2024/09/eurosys25-final999.pdf.

Yu, S., Shu, J., Ni, Y., Qian, K., Li, X., Wang, Y., Zhang, J., Xu, Z., Yang, S., Zhu, L., Zhai, E., Lu, Q., Xing, J., Lu, Y., Jin, X., Liu, X., and Xu, H. Pythia: Exploiting workflow predictability for efficient agent-native LLM serving, 2026. URL https://arxiv.org/abs/2604.25899.

Yuan, Y., Nayak, A., Kundu, S., and Talati, N. Agentic AI workload characteristics. In *2026 IEEE International Symposium on Workload Characterization (IISWC)*, 2026. URL https://arxiv.org/abs/2605.26297.

Zhang, R., Kim, C., Feng, S., Du, K., Liu, Y., Zhong, Y., Ching, C.-W., Jiang, J., and Hu, L. Learning agent execution for kv-cache management in agentic serving, 2026.

Zheng, H., Fu, F., Wu, J., Yuan, B., Zhang, Y., Wang, H., Zhu, Y., Yan, X., and Jiang, J. Efficient serving for dynamic agent workflows with prediction-based KV-cache management, 2026. URL https://arxiv.org/abs/2605.06472.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024.